

Mutex ի՞նչ է — Խորը ուղեցույց

C++ Concurrency presentation

Այս ուղեցույցը մանրամասն բացատրում է mutex-ը՝ ինչ է, ինչպես է լուծում race condition-ը, lock/unlock, lock_guard, unique_lock, օրինակներով և հարցազրույցի հարցերով:

1. Ի՞նչ է Mutex-ը

Mutex = **M**UTual **E**Xclusion. Սինխրոնիզացիայի մեխանիզմ, որ ապահովում է, որ **միայն մեկ thread** միանգամից մուտք ունի shared resource-ին (critical section):

Mutex = «բանալի» -> միայն մեկ thread կարող է «կախել» այն միանգամից
lock() -> վերցնում բանալին (եթե զբաղված է, սպասում)
unlock() -> վերադարձնում բանալին

```
#include <mutex>
std::mutex mtx;
int counter = 0;

void increment() {
    mtx.lock();           // critical section սկսվում
    counter++;
    mtx.unlock();         // critical section ավարտվում
}
```

2. Խնդիրը առանց Mutex (Race Condition)

```
int counter = 0;
void increment() {
    for (int i = 0; i < 1000000; i++)
        counter++;    // 3 քայլ: read, +1, write -> thread-ներ խառնվում
}
// 2 thread -> counter != 2000000 (race condition)
```

`counter++` -ը **atomic** չէ (read-modify-write): Երկու thread միասին -> արժեքներ կորչեն:

3. Լուծումը Mutex-ով

```
std::mutex mtx;
int counter = 0;

void increment() {
    for (int i = 0; i < 1000000; i++) {
        mtx.lock();
        counter++;    // միայն մեկ thread միանգամից
        mtx.unlock();
    }
}
// հիմա counter == 2000000 (ճիշտ)
```

Mutex-ը ապահովում է, որ `counter++` -ը կատարվի անբաժանելի (մեկ thread միանգամից):

4. lock_guard (RAII, ՆԱԽԸՆՏՐԵԼԻ)

Ձեռքով `lock()` / `unlock()` -ը վտանգավոր է (exception -> unlock չի հասնի -> deadlock):

`lock_guard` -ը RAII է:

```
std::mutex mtx;
int counter = 0;

void increment() {
    std::lock_guard<std::mutex> lock(mtx);    // ctor -> lock
    counter++;
}    // dtor -> unlock (ավտոմատ, նույնիսկ exception-ի դեպքում)
```

`lock_guard` -ը. lock ctor-ում, unlock dtor-ում -> չկա «մոռացած unlock»:

5. unique_lock (ավելի ճկուն)

```
std::unique_lock<std::mutex> lock(mtx);
// ...
lock.unlock();    // ձեռքով unlock կարող
lock.lock();      // նորից lock
// condition_variable-ի հետ օգտագործվում
```

lock_guard -> պարզ, lock ctor / unlock dtor (չի բացվում)
 unique_lock -> ճկուն (ձեռքով lock/unlock, defer, condition_variable)

6. Mutex-ի տեսակները

```
std::mutex                // սովորական
std::recursive_mutex      // նույն thread-ը կարող է մի քանի անգամ lock անել
std::timed_mutex          // try_lock_for (timeout-ով)
std::shared_mutex         // shared (read) + exclusive (write) -- C++17
```

7. Կարևոր կանոններ

- Միշտ `lock_guard/unique_lock` (ոչ ձեռքով `lock/unlock`)
- `Critical section`-ը պահի ՓՈՔՐ (ավելի քիչ `blocking`)
- Նույն կարգով `lock` անի մի քանի `mutex` (`deadlock`-ից խուսափել)
- Չկա `lock`-ի մեջ երկար `operation` (I/O)

8. Հարցազրույցի հարցեր և պատասխաններ

Mutex ինչ է:

Mutual Exclusion -- սինխրոնիզացիայի մեխանիզմ, միայն մեկ `thread` միանգամից մուտք `shared resource`-ին:

Ի՞նչ խնդիր է լուծում:

Race condition -- `shared data`-ի անվտանգ մուտք:

lock() / unlock():

lock վերցնում բանալին (սպասում եթե զբաղված); *unlock* վերադարձնում:

lock_guard ինչու նախընտրելի:

RAII -- `lock ctor / unlock dtor`; *exception-safe*, չկա մոռացում:

lock_guard vs unique_lock:

lock_guard պարզ (չի բացվում); *unique_lock* ճկուն (ձեռքով `lock/unlock`, `condition_variable`):

recursive_mutex ինչ է:

Նույն `thread`-ը կարող է մի քանի անգամ `lock` անել (սովորական `mutex`-ը -> `deadlock`):

Mutex-ի վտանգը:

Deadlock (եթե սխալ օգտագործես), performance (blocking):

Ամփոփում (Cheat Sheet)

MUTEX = Mutual Exclusion -> միայն մեկ thread միանգամից critical section-ում

lock() / unlock() -> ձեռքով (ՎՏԱՆԳԱՎՈՐ)

lock_guard -> RAII (lock ctor/unlock dtor) ԼԱԽՆՆՏՐԵԼԻ

unique_lock -> ճկուն (ձեռքով lock/unlock, condition_variable)

ՏԵՍԱԿՆԵՐ: mutex | recursive_mutex | timed_mutex | shared_mutex (C++17)

ԼՈՒԾՈՒՄ: race condition (shared data անվտանգ մուտք)

ԿԱՆՈՆ: lock_guard օգտագործիր | critical section փոքր | նույն կարգով lock (deadlock խուսափել)